

BCS613A – Blockchain Technology

Module 1 | Exam-Ready Answers

Q.01(a) Define Blockchain. Explain the network view of Blockchain with a neat diagram.

Definition of Blockchain:

Blockchain is a peer-to-peer distributed ledger that is cryptographically secure, append-only, immutable (extremely hard to change), and updateable only via consensus or agreement among peers.

From a business perspective, blockchain is a platform where peers can exchange values using transactions **without the need for a central trusted arbitrator**.

Key Properties:

- **Distributed Ledger** – Data is stored across many nodes, not in one central place.
- **Cryptographically Secure** – Each block is linked using cryptographic hash functions.
- **Append-Only** – New data can only be added; old data cannot be deleted.
- **Immutable** – Once recorded, data is practically impossible to alter.
- **Consensus-Based** – Updates require agreement among peers (e.g., PoW, PoS).

Network View of Blockchain:

Blockchain can be thought of as a layer running on top of the Internet, similar to how SMTP, HTTP, or FTP run on TCP/IP. The layered architecture is shown below:

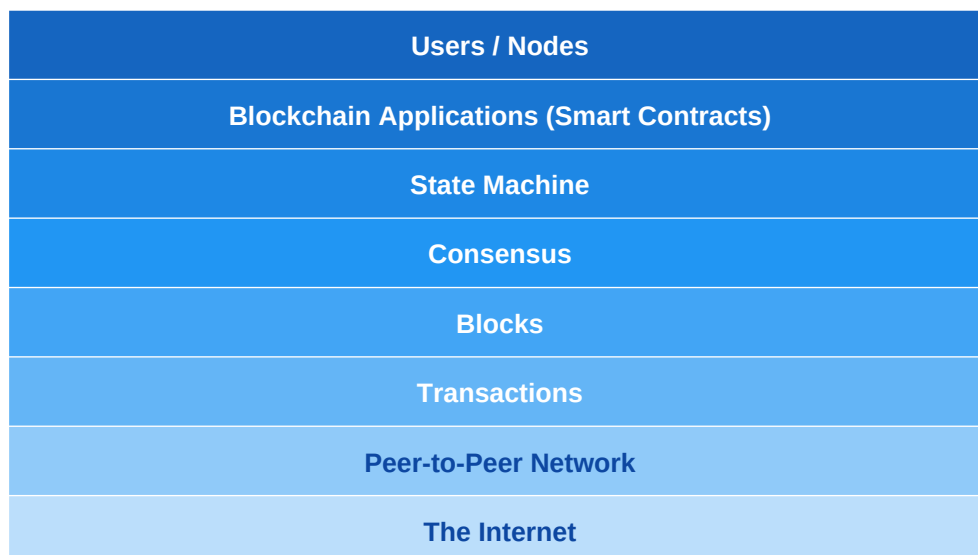


Fig: Network (Layered) View of Blockchain

Architectural Layers Explained:

- **Network Layer** – Internet / TCP-IP (bottom-most layer).
- **P2P Layer** – Gossip protocol, routing, flooding protocols.
- **Cryptography Layer** – Public key cryptography, digital signatures, hash functions.

- **Consensus Layer** – PoW / PoS / PBFT protocols.
 - **Execution Layer** – Virtual machines, blocks, transactions.
 - **Application Layer** – Smart contracts, DApps, DAOs, Autonomous Agents.
 - **Users/Nodes** – Interact at the top level.
-

Q.01(b) List and explain the generic elements of Blockchain.

A blockchain has several generic elements that form its core structure:

- 1. Addresses:** Addresses are unique identifiers used in a transaction to denote senders and recipients. An address is usually a public key or derived from a public key. Best practice: generate a new address for each transaction to avoid linking transactions to the same owner.
 - 2. Transaction:** A transaction is the fundamental unit of a blockchain. It represents a transfer of value from one address to another. Transactions are verified and grouped into blocks.
 - 3. Block:** A block is composed of multiple transactions along with: Previous block hash (hash pointer), Timestamp, Nonce. The first block is called the Genesis Block (has no previous hash).
 - 4. Peer-to-Peer Network:** A network topology where all peers can communicate with each other, send and receive messages directly without a central server.
 - 5. Scripting / Programming Language:** Performs operations on transactions. Transaction scripts are predefined command sets for transferring tokens. Turing-complete languages are desirable but raise security concerns.
 - 6. Virtual Machine:** An extension of a transaction script. Allows Turing-complete code (smart contracts) to run on a blockchain. Examples: Ethereum Virtual Machine (EVM), Chain Virtual Machine (CVM).
 - 7. State Machine:** Blockchain can be viewed as a state transition mechanism: a state is modified from its initial form to the next as a result of transaction execution and validation by nodes.
 - 8. Nodes:** A node performs various functions: propose and validate transactions, mining to facilitate consensus, simple payment verification (lightweight nodes), validators, etc.
 - 9. Smart Contracts:** Programs that run on top of the blockchain encapsulating business logic. They execute automatically when certain conditions are met. Not available on all blockchains but a very desirable feature.
-

Q.02(a) Define Consensus. List the requirements and types of Consensus mechanism.

Definition:

Consensus is a process of agreement between distrusting nodes on a final state of data. When multiple nodes in a distributed system need to agree on a single value, it is called **Distributed Consensus**.

Requirements of Consensus Mechanism:

- **Agreement:** All honest nodes decide on the same value.

- **Termination:** All honest nodes eventually reach a decision and stop.
- **Validity:** The agreed value must be the one initially proposed by at least one honest node.
- **Fault Tolerant:** The algorithm must work even in the presence of faulty or malicious (Byzantine) nodes.
- **Integrity:** No node makes a decision more than once in a single consensus cycle.

Types of Consensus Mechanism:

Type	Description
Byzantine Fault Tolerance (BFT)-based	No compute-intensive operations. Nodes publish signed messages; agreement is reached by majority.
Leader-based (Nakamoto Consensus)	Nodes compete for leader election. The winner proposes the final value. Examples: Bitcoin, Ethereum.
Proof of Work (PoW)	Requires proof that enough computational resources (hash puzzle) have been spent.
Proof of Stake (PoS)	Node's chance of proposing the next block depends on how much it has invested (stake).
Delegated PoS (DPoS)	Nodes with stake delegate transaction validation to other nodes by voting. Used in BitShares.
Practical BFT (PBFT)	Achieves state machine replication providing tolerance against Byzantine nodes. Used in Hyperledger Fabric.
Deposit-based	Nodes must put in a security deposit before proposing a block.
Federated Byzantine	Used in Stellar; nodes keep a group of trusted peers and propagate only transaction they agree on.

Q.02(b) List and explain the Consensus Algorithms or mechanisms.

Consensus algorithms are classified into two broad categories:

- **Proof-based / Leader-based (Nakamoto Consensus)** – A leader is elected to propose the final value.
- **BFT-based** – Traditional voting-round approach.

1. Proof of Work (PoW):

Used in Bitcoin. Requires solving a complex hash puzzle (finding a nonce) before proposing a block. The solved hash proves computational effort. It is the only mechanism proven successful against Sybil attacks. It is energy-intensive.

2. Proof of Stake (PoS):

First introduced by Peercoin; used in Ethereum. A node's probability of proposing the next block depends on its coin age (coins held × time). More stake = higher chance of being selected. Energy-efficient compared to PoW.

3. Delegated Proof of Stake (DPoS):

Used in BitShares. Nodes with stake vote to delegate transaction validation to trusted nodes. More democratic and faster than standard PoS.

4. Proof of Elapsed Time (PoET):

Introduced by Intel. Uses Trusted Execution Environment (TEE) and Intel SGX hardware to guarantee a random, secure wait time for leader election. Energy efficient and random.

5. Practical Byzantine Fault Tolerance (PBFT):

Proposed by Castro and Liskov (1999). Achieves state machine replication and tolerates Byzantine nodes. Works well when 2/3 of nodes are honest. Used in Hyperledger Fabric.

6. Federated Byzantine Agreement (FBA):

Used in Stellar Consensus Protocol. Each node keeps a group of publicly trusted peers. A transaction is propagated only if validated by the majority of trusted peers.

7. Paxos:

Introduced by Leslie Lamport (1989). Nodes are assigned roles: Proposer, Acceptor, Learner. Consensus is achieved when the majority of nodes (replicas) agree.

8. RAFT:

An alternative to Paxos. Each node is in one of three states: Follower, Candidate, or Leader. A Candidate becomes Leader after receiving enough votes. All changes go through the Leader.

9. Reputation-based Mechanisms:

A leader is elected based on its reputation built over time in the network, which can be determined by voting from other members.

10. Deposit-based Consensus:

Nodes must place a security deposit before proposing a block. This discourages malicious behavior as deposits can be forfeited for dishonest actions.

Q.02(c) Explain benefits and limitations of Blockchain.

Benefits of Blockchain:

- **Decentralization:** No trusted third party or intermediary is needed. Consensus mechanism validates transactions.
- **Transparency and Trust:** Everyone on the network can see the data, building trust among participants.
- **Immutability:** Once data is written, it is practically impossible to alter (changing it would require re-mining all subsequent blocks).
- **High Availability:** Based on thousands of nodes in a P2P network; even if some nodes go offline, the system continues to function.
- **Highly Secure:** All transactions are cryptographically secured, providing integrity, non-repudiation, and authentication.
- **Simplification:** Acts as a single shared ledger for multiple organizations, reducing complexity of maintaining separate databases.
- **Faster Dealings:** Financial settlements are quicker as a single agreed-upon version of data is already available.
- **Cost Saving:** Eliminates fees paid to clearing houses or trusted third parties.

Limitations of Blockchain:

- **Scalability:** Block size is fixed (1 MB in Bitcoin), limiting the number of transactions per block. Cannot scale easily.
- **Energy Consuming:** PoW requires massive computational power. By 2018, about 0.3% of world electricity was used for blockchain verification.
- **Time Consuming:** Mining requires computing many nonce values, making block addition slow for industrial use.
- **Immaturity:** Blockchain is still a relatively young technology. Many people lack confidence and are not ready to adopt it widely.
- **Storage Issues:** Blockchain is stored on every node. As transactions grow, storage requirements become significant.
- **Legal / Regulatory Issues:** Some countries have banned blockchain applications (e.g., cryptocurrency) due to environmental and legal concerns.
- **Privacy Concerns:** Public blockchains are transparent by design; confidentiality is difficult to achieve.
- **Regulation:** Faces challenges with financial institutions; regulatory frameworks are still evolving.

Q.1(a) Define Blockchain and explain Centralized, Decentralized and Distributed systems with a suitable diagram.

Blockchain is a peer-to-peer distributed ledger that is cryptographically secure, append-only, immutable, and updateable only via consensus among peers.

1. Centralized Systems:

All processing and data storage is done on a **single central server**. Clients (nodes) rely on the server for computation. The server acts as the hub for all requests.

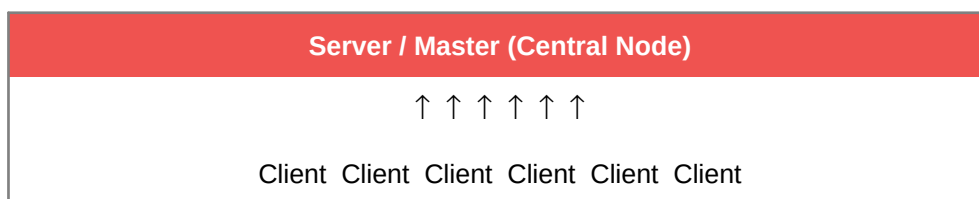


Fig: Centralized System

Key features: Single point of control, easy to manage, but single point of failure, limited scalability.

Examples: ERP systems, CRM systems, Email servers, Banking systems.

2. Decentralized Systems:

Multiple nodes share control without a single central authority. Each node operates independently but collaborates. Enhances fault tolerance, scalability, and resilience.

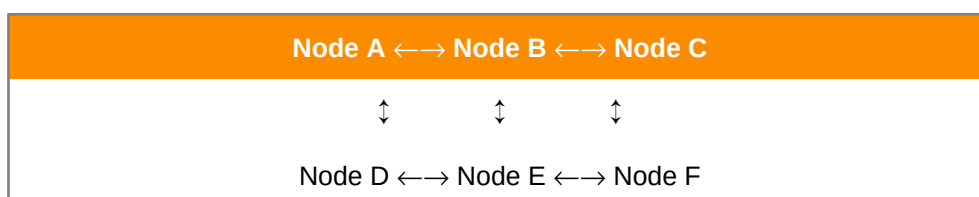


Fig: Decentralized System – Multiple control points

Key features: Distributed control, fault tolerant, scalable, no single point of failure.

Examples: Bitcoin, Ethereum, BitTorrent, DeFi platforms.

3. Distributed Systems:

Multiple independent nodes work together to achieve a common goal. They appear as a single system to the end user. Aim to improve performance, reliability, scalability, and resource sharing.

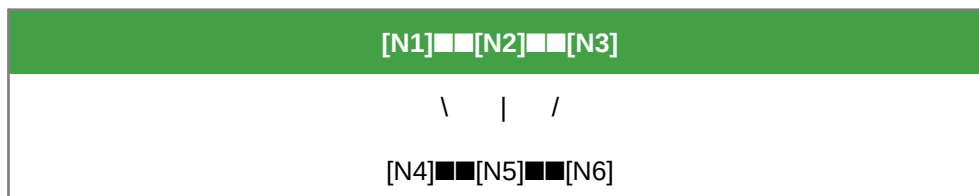


Fig: Distributed System – All nodes are equal peers

Key features: Geographical distribution, resource sharing, concurrency, scalability, fault tolerance, transparency.

Examples: AWS, CDNs (Cloudflare), Google Spanner, Netflix microservices.

Comparison Table:

Aspect	Centralized	Decentralized	Distributed
Control	Single central node	Multiple independent nodes	Shared among all nodes
Failure Risk	High (single point)	Low	Low (fault tolerant)
Scalability	Limited	Moderate	High
Performance	Good initially, degrades under load		High (parallel)
Examples	ERP, Banking	Bitcoin, P2P	AWS, CDN

Q.1(b) Explain the generic structure of a block with a diagram.

A **block** is simply a selection of transactions bundled together logically. It is the fundamental unit of a blockchain. Its size is variable depending on the type and design of the blockchain.

Structure of a Block:

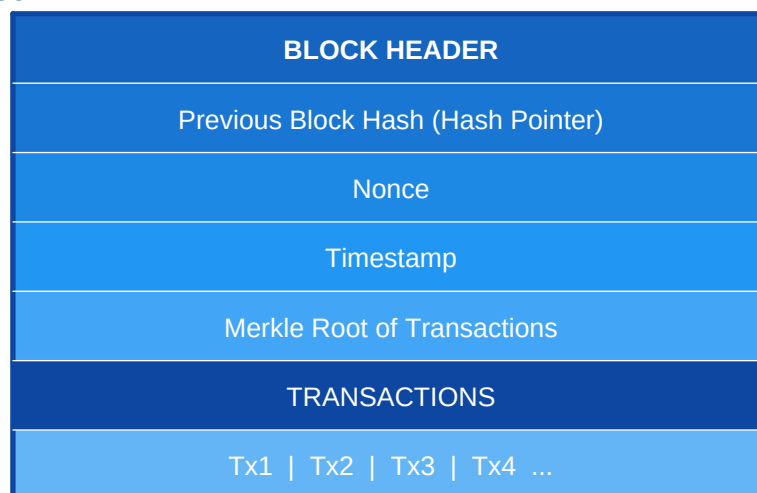


Fig: Structure of a Block

Explanation of Block Components:

- **Previous Block Hash:** A hash pointer linking this block to the previous block, forming the chain. Absent only in the Genesis Block.
- **Nonce:** A number used in mining – miners iterate nonce values until the hash meets the difficulty target.
- **Timestamp:** Records the time when the block was created.
- **Merkle Root:** A single hash summarizing all transactions in the block, enabling efficient verification.
- **Transactions:** The actual data (transfers of value) stored in the block. Multiple transactions are bundled together.
- **Genesis Block:** The very first block in the blockchain, hardcoded with no reference to a previous block.

Q.1(c) Discuss the growth of the blockchain with an Architecture / Network view with a neat diagram.

Blockchain technology has grown across three main generations and continues to evolve.

Tiers / Generations of Blockchain:

- **Blockchain 1.0 (Cryptocurrency):** Introduced with Bitcoin (2009). Used for cryptocurrencies and digital payments. Includes Bitcoin, Litecoin, and other altcoins. Core applications: payments.
- **Blockchain 2.0 (Smart Contracts & Finance):** Used by financial services. Introduced smart contracts (Ethereum, 2015). Includes derivatives, options, swaps, bonds. Beyond just currency.
- **Blockchain 3.0 (General Purpose):** Applications beyond finance: government, health, media, arts, justice. Includes DeFi platforms, NFTs, and enterprise blockchains like Hyperledger.
- **Blockchain X (Future Vision):** A public blockchain service available to everyone like a search engine. Features general-purpose rational agents making decisions autonomously on behalf of humans.

Network / Architectural View:

USERS / NODES	
APPLICATIONS	Smart Contracts DApps DAOs Autonomous Agents
EXECUTION	Virtual Machines Blocks Transactions
CONSENSUS	State Machine Replication PoW PoS PBFT
CRYPTOGRAPHY	Public Key Crypto Digital Signatures Hash Functions
P2P NETWORK	Gossip Protocol Routing Flooding Protocols
NETWORK	The Internet / TCP-IP

Fig: Architectural View of Blockchain

Q.2(a) Illustrate the Byzantine General Problem with a suitable example.

History:

In 1982, the Byzantine General's Problem was invented by **Leslie Lamport, Robert Shostak, and Marshall Pease**. It is a game theory problem describing how decentralized parties face difficulty reaching consensus without a trusted central party.

The Problem:

The Byzantine army is divided into many battalions, each led by a general. The generals must coordinate a joint attack on a city by communicating only through messengers. The challenge: **some generals (or messengers) may be traitors** who try to sabotage the plan by sending false or conflicting messages.

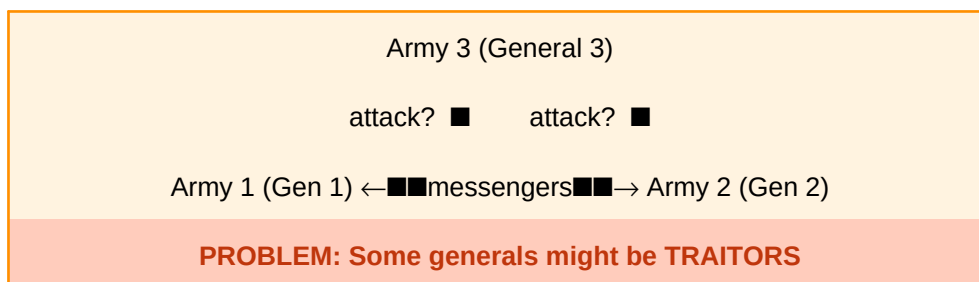


Fig: Byzantine Generals Problem

Example:

Suppose 3 generals must decide to ATTACK or RETREAT. General 1 says attack, General 2 says attack, but General 3 (traitor) tells Gen 1 'attack' and tells Gen 2 'retreat'. This confusion can lead to only part of the army attacking, resulting in defeat.

How Bitcoin Solves It:

- **Blockchain as Agreement:** The blockchain is the untampered agreement all loyal generals must agree on.
- **Proof of Work:** Bitcoin uses PoW so that creating false blocks requires enormous computational resources – making fraud impractical.
- **Counterfeit Resistant:** Miners must present a valid hash (proof of work). Dishonest blocks are immediately rejected.
- **Provable:** Once a block is added, it is nearly impossible to erase, making history immutable.
- **Trust-free:** Every node independently verifies all information; no trust in other nodes is needed.
- **Byzantine Fault Tolerance (BFT):** When 2/3 of the network agrees, the system is considered BFT. Solved practically in 1999 (PBFT) and in Bitcoin (2009).

Q.2(b) Describe the CAP theorem in Blockchain.

Definition:

The **CAP Theorem** (also known as **Brewer's Theorem**) was introduced by Eric Brewer in 1998 and proved by Seth Gilbert and Nancy Lynch in 2002. It states that a distributed system **cannot simultaneously provide all three** of the following properties:

Property	Symbol	Description
Consistency	C	All nodes have the same, latest copy of data at the same time.
Availability	A	A system is always up, accepting requests and responding without failures.
Partition Tolerance	P	The system continues to operate even if some nodes or network links fail.

The theorem states: A distributed system can only guarantee 2 out of 3 properties at any time.

CAP Theorem and Blockchain:

- Blockchain appears to violate CAP theorem but it does not.
- In blockchain, **Consistency (C) is sacrificed** in favor of Availability (A) and Partition Tolerance (P).
- Consistency is achieved **over time** – this is called **Eventual Consistency**.
- Eventual consistency means all nodes will eventually agree on the state of data after validations from multiple nodes.
- In Bitcoin, **mining** is the process that achieves eventual consistency using PoW.
- Mining can be defined as a process used to add more blocks to the blockchain, facilitating consensus.
- Usually **6 confirmations** are required in the Bitcoin network to consider a transaction final.

Conclusion: Blockchain is an AP system (Available + Partition Tolerant) with eventual consistency – it does not violate CAP; it trades immediate consistency for availability.

Q.2(c) Why are Consensus Algorithms needed in Blockchain? Explain in detail.

Why Consensus is Needed:

In a distributed blockchain network, there is **no central authority** to validate or approve transactions. Thousands of nodes hold copies of the ledger. The key challenge is: **how do all these nodes agree on the same version of truth?** This is solved by consensus algorithms.

- 1. Prevent Double Spending:** Without consensus, a malicious user could spend the same coin twice on different nodes. Consensus ensures only one valid transaction is accepted network-wide.
- 2. Establish Trust Without Central Authority:** Consensus allows strangers on the internet to trust the blockchain without needing a bank or government to verify transactions.
- 3. Achieve State Machine Replication:** Blockchain is a state machine. Consensus ensures all nodes transition to the same next state after each transaction.
- 4. Handle Byzantine (Faulty) Nodes:** Some nodes may be malicious or faulty. Consensus algorithms like BFT ensure the network continues correctly even with such nodes.
- 5. Ensure Data Integrity and Immutability:** Once consensus is reached on a block, altering it would require redoing the consensus for all subsequent blocks – practically impossible.
- 6. Decentralization:** Consensus enables decentralization by removing the need for a central trusted party, replacing it with a mathematical agreement protocol followed by all nodes.

7. Sybil Attack Prevention: PoW consensus prevents Sybil attacks (creating fake identities) because valid blocks require massive computational effort.

How Consensus Works in Bitcoin:

Step 1: A node starts a transaction by signing it with its private key.

Step 2: The transaction is flooded to peers using the Gossip protocol for validation.

Step 3: Once validated, the transaction is included in a block and propagated to the network.

Step 4: The new block is linked cryptographically to the previous block via hash pointer.

Step 5: Transactions are reconfirmed every time a new block is created. 6 confirmations = final.

Conclusion: *Without consensus algorithms, blockchain would be just a shared database with no way to prevent fraud, resolve disagreements, or maintain a single source of truth across thousands of untrusted nodes.*